

Deep Research: Avatrada 57 Topic 036

Engineering Report: Dual-Feed Market Data Architecture (ThetaData/IBKR)

Executive Summary

This report provides a detailed engineering analysis of the specified dual-feed market data architecture. The system is designed to leverage ThetaData for low-latency alpha signal generation and Interactive Brokers (IBKR) for data validation and execution. This hybrid approach aims to combine the speed of a specialized data provider with the reliability and execution capabilities of a major brokerage, creating a robust platform for automated trading.

The analysis covers the system's technical deconstruction, a comprehensive implementation strategy in Python using modern asynchronous patterns, and a critical analysis of potential failure modes, edge cases, and optimizations. The core challenges addressed are managing concurrent data streams, mapping disparate symbology formats, and synchronizing data from two sources with different latency characteristics.

1. Technical Deconstruction

The proposed system is a classic example of a "speed vs. safety" architecture. It decouples the time-sensitive task of signal generation from the mission-critical tasks of validation and execution.

1.1 Core Components

1. **ThetaData Feed Handler:** A dedicated, lightweight component responsible for a single task: subscribing to ThetaData's WebSocket feed.

Its goal is to ingest raw market data (quotes, trades) with the lowest possible latency. This data is the primary input for the alpha model.

2. **IBKR Feed Handler:** This component connects to the IBKR TWS/Gateway API. Its primary role is to provide a secondary, trusted source of market data. This data is not used for the initial signal generation but for validating the signals generated from the ThetaData feed.
3. **Symbol Mapping Service:** A crucial translation layer. Data providers rarely use identical symbology, especially for derivatives. This service is responsible for converting a symbol from the ThetaData format (e.g., `SPXW231220C48000000`) into a contract object that the IBKR API can understand.
4. **Synchronization & Processing Core:** The heart of the system. It receives low-latency data from the ThetaData handler and, upon generating a potential signal, queries the state maintained by the IBKR handler to validate the price. It enforces rules about data staleness and price deviation before passing a signal to the execution logic.
5. **Signal Generation & Execution Logic:** The business logic layer. The signal generator consumes ThetaData ticks to identify trading opportunities. The execution module takes validated signals from the processing core and places orders via the IBKR API.

1.2 Data Flow Architecture

The data flows in a directed, staged manner to ensure both speed and correctness.

```
graph TD
    A[ThetaData WebSocket] -->|Raw Ticks| B[ThetaData Handler];
    B -->|Parsed Data| C[Synchronization Queue];

    D[IBKR TWS/Gateway] -->|Streaming Market Data| E[IBKR Handler];
    E -->|IBKR Contract & Price| F[Shared State: IBKR Price Cache];

    C -->|ThetaData Tick| G[Signal & Validation Processor];
    F -->|Get Latest Price| G;
```

```

G -->|Is Valid?| H{Decision Point};
H -- Yes --> I(Execution Engine);
H -- No --> J(Log/Discard);

I -->|Place Order| D;

subgraph "Low-Latency Path (Alpha)"
    A
    B
    C
end

subgraph "Validation & Execution Path"
    D
    E
    F
    I
end

subgraph "Central Logic"
    G
    H
    J
end

```

Workflow:

1. The **ThetaData Handler** receives a tick and immediately places it into a high-speed, in-memory queue.
2. The **IBKR Handler** independently maintains a real-time price cache (e.g., a Python dictionary) for all monitored symbols, updated via its own streaming data subscription.
3. The **Signal & Validation Processor** consumes a tick from the queue. It first runs its alpha logic.
4. If a signal is generated, it looks up the corresponding IBKR symbol via the **Symbol Mapping Service**.
5. It then retrieves the most recent price from the **IBKR Price Cache**.

6. It validates the signal by comparing the ThetaData price against the IBKR price and checking the timestamp of the IBKR data to ensure it's not stale.
 7. If validation passes, the signal is forwarded to the **Execution Engine**.
-

2. Implementation Strategy

We will architect the system using Python's `asyncio` library, which is superior to traditional `threading` for I/O-bound applications like managing multiple network streams. `ib_insync` is the recommended library for IBKR integration due to its native `asyncio` support.

2.1 Libraries and Setup

- `asyncio` : For managing concurrent tasks (feed handlers, processor).
- `websockets` : For connecting to the ThetaData WebSocket feed.
- `ib_insync` : For connecting to IBKR TWS/Gateway and handling data/execution.
- `logging` : For robust event logging.

2.2 Symbology Mapping

This is the most critical translation component. ThetaData uses a concise string format, while IBKR requires a structured `Contract` object. We need a parser that can reliably perform this conversion.

ThetaData Option Format: `[ROOT]YYMMDD[C/P][STRIKE_PRICE * 1000]`

Example: `SPXW231220C48000000`

IBKR `ib_insync.Option` **Object:** Requires `symbol`, `lastTradeDateOrContractMonth`, `strike`, `right`, `exchange`.

```
# implementation/symbology_mapper.py
import re
from ib_insync import Contract, Option

def map_thetadata_to_ibkr(theta_symbol: str) -> Contract:
```

```

"""
    Parses a ThetaData option symbol and returns an ib_insync Contract object.
    Example: SPXW231220C4800000 -> Option('SPXW', '20231220', 4800, 'C',
'SMART')
"""
    # Regex to capture the components of the symbol
    # Handles roots like SPX, SPXW, VIX, etc.
    match = re.match(r"([A-Z]+)(\d{6})([CP])(\d+)", theta_symbol)

    if not match:
        raise ValueError(f"Invalid ThetaData symbol format: {theta_symbol}")

    root, date_str, right_char, strike_milli = match.groups()

    # Format the date correctly for IBKR
    # Theta: YYMMDD -> IBKR: YYYYMMDD
    full_date_str = f"20{date_str}"

    # Convert strike from integer * 1000 to float
    strike = float(strike_milli) / 1000.0

    # Map 'C'/'P' to 'Call'/'Put'
    right = 'C' if right_char == 'C' else 'P'

    # NOTE: Exchange and currency are often context-dependent.
    # For SPX/SPXW, 'CBOE' is specific, but 'SMART' is a good default for
routing.
    # The underlying symbol for options might differ from the root (e.g. SPXW
root -> SPX underlying)
    # This logic may need refinement based on the specific assets traded.
    underlying_symbol = 'SPX' if 'SPX' in root else root

    contract = Option(
        symbol=underlying_symbol,
        lastTradeDateOrContractMonth=full_date_str,
        strike=strike,
        right=right,
        exchange='SMART', # Use SMART routing
        currency='USD'
    )

```

```

    return contract

# Example Usage:
# spxw_contract = map_thetadata_to_ibkr("SPXW231220C4800000")
# print(spxw_contract)

```

2.3 Concurrency and Synchronization Strategy

We will use an `asyncio.Queue` for passing data from the fast producer (ThetaData) to the consumer (Processor). A shared dictionary will serve as the state cache for IBKR prices.

```

# implementation/main_architecture.py
import asyncio
import websockets
from ib_insync import IB, util, Ticker
from typing import Dict

# --- Shared State ---
# Queue for incoming low-latency data
theta_data_queue = asyncio.Queue()
# Cache for validation data. Key: IBKR Contract, Value: Ticker
ibkr_price_cache: Dict[Contract, Ticker] = {}
# Lock for safely accessing the shared cache if complex operations are needed
cache_lock = asyncio.Lock()

# --- 1. ThetaData Handler (Producer) ---
async def thetadata_handler(uri):
    async with websockets.connect(uri) as websocket:
        # Authenticate, subscribe to symbols, etc.
        # ...
        async for message in websocket:
            # Parse the message (e.g., JSON)
            parsed_data = parse_theta_message(message)
            await theta_data_queue.put(parsed_data)

# --- 2. IBKR Handler (State Maintainer) ---
def on_ibkr_tick(ticker: Ticker):

```

```

    """Callback to update the price cache."""
    # This function is synchronous, called by the ib_insync event loop
    # No need for async lock here as dict assignment is atomic in Python
    ibkr_price_cache[ticker.contract] = ticker
    # print(f"IBKR Cache Updated: {ticker.contract.localSymbol} ->
{ticker.last}")

async def ibkr_handler(ib: IB, contracts_to_watch: list):
    for contract in contracts_to_watch:
        ib.reqMktData(contract, '', False, False)
        ib.pendingTickersEvent += on_ibkr_tick
        # Keep the handler alive to process events
        while ib.isConnected():
            await asyncio.sleep(1)
        ib.pendingTickersEvent -= on_ibkr_tick

# --- 3. Signal & Validation Processor (Consumer) ---
async def signal_processor(ib: IB):
    STALENESS_THRESHOLD_SECONDS = 0.5 # 500ms
    PRICE_DEVIATION_TOLERANCE = 0.05 # 5 cents

    while True:
        theta_tick = await theta_data_queue.get()

        # 1. Generate Signal (Alpha Logic)
        if not is_signal(theta_tick):
            continue

        # 2. Map Symbolology
        try:
            ibkr_contract = map_thetadata_to_ibkr(theta_tick['symbol'])
        except ValueError as e:
            logging.error(e)
            continue

        # 3. Validate
        ibkr_ticker = ibkr_price_cache.get(ibkr_contract)

        if not ibkr_ticker:
            logging.warning(f"No IBKR data for {ibkr_contract.localSymbol} to

```

```

validate signal.")
    continue

    # 3a. Staleness Check
    now = datetime.now(timezone.utc)
    time_diff = (now - ibkr_ticker.time).total_seconds()
    if time_diff > STALENESS_THRESHOLD_SECONDS:
        logging.warning(f"Stale IBKR data for {ibkr_contract.localSymbol}:
{time_diff:.2f}s old.")
        continue

    # 3b. Price Check (using Midpoint or Last Price)
    theta_price = (theta_tick['bid'] + theta_tick['ask']) / 2
    ibkr_price = (ibkr_ticker.bid + ibkr_ticker.ask) / 2

    if abs(theta_price - ibkr_price) > PRICE_DEVIATION_TOLERANCE:
        logging.warning(f"Price deviation for {ibkr_contract.localSymbol}:
Theta={theta_price}, IBKR={ibkr_price}")
        continue

    # 4. Execute
    logging.info(f"VALIDATED SIGNAL: Placing order for
{ibkr_contract.localSymbol}")
    # order = MarketOrder(...)
    # trade = ib.placeOrder(ibkr_contract, order)
    # ...

# --- Main Application Entrypoint ---
async def main():
    # Setup IB connection
    ib = IB()
    await ib.connectAsync('127.0.0.1', 7497, clientId=10)

    # Define symbols and map them
    theta_symbols = ["SPXW231220C4800000", "SPXW231220P4700000"]
    ibkr_contracts = [map_thetadata_to_ibkr(s) for s in theta_symbols]

    # Start all concurrent tasks
    theta_task = asyncio.create_task(thetadata_handler("wss://..."))
    ibkr_task = asyncio.create_task(ibkr_handler(ib, ibkr_contracts))

```



```
processor_task = asyncio.create_task(signal_processor(ib))

await asyncio.gather(theta_task, ibkr_task, processor_task)

if __name__ == "__main__":
    util.patchAsyncio() # Patch to make ib_insync work with asyncio event loop
    try:
        asyncio.run(main())
    except (KeyboardInterrupt, SystemExit):
        logging.info("System shutdown requested.")
```

3. Critical Analysis

3.1 Potential Failure Modes & Mitigations

- **Feed Disconnection:** Either the ThetaData WebSocket or the IBKR TWS connection can drop.
 - **Mitigation:** Implement robust reconnection logic in both handlers. Use an exponential backoff strategy to avoid overwhelming the servers. The system should enter a "safe mode" (no new positions) when either feed is down.
- **Symbology Mapping Failure:** A new or unexpected symbol format from ThetaData could break the `map_thetadata_to_ibkr` function.
 - **Mitigation:** The function must have comprehensive error handling (`try...except`). Failed mappings should be logged aggressively for immediate review. A "dead-letter queue" could be implemented for symbols that fail to parse, allowing for manual intervention.
- **Synchronization Latency (Staleness):** The core assumption is that the IBKR price is a recent, valid snapshot. If the IBKR feed is delayed, we might validate a signal against old data.
 - **Mitigation:** The staleness check (`STALENESS_THRESHOLD_SECONDS`) is critical. This value must be tuned based on observed latencies. If data

is consistently stale, it indicates a performance issue with the IBKR connection or the local machine.

- **"Bad Ticks" / Erroneous Data:** One provider might send an anomalous price (e.g., a price of zero or an extremely wide bid-ask spread).
 - **Mitigation:** The price deviation check helps catch discrepancies between providers. Additional sanity checks should be added within each feed handler before data is even passed to the core logic (e.g., `if bid <= 0 or ask <= 0 or ask < bid: discard_tick()`).

3.2 Edge Cases

- **Trading Halts / Market Pauses:** During a halt, ThetaData might continue to send quotes while IBKR reports the instrument as "Not Tradeable."
 - **Analysis:** The execution logic must check the `tradeable` status of an instrument from IBKR before placing an order. The IBKR ticker object often contains market state information that should be used.
- **Partial Fills & Order Management:** The current design focuses on signal validation. A complete system needs a sophisticated order management module to handle partial fills, cancellations, and position tracking, all synchronized with IBKR's state.
- **Clock Synchronization:** The system relies on timestamps for staleness checks. If the local machine's clock drifts significantly from the providers' clocks, these checks can become unreliable.
 - **Analysis:** Use NTP (Network Time Protocol) to keep the system clock synchronized. When comparing timestamps, always use timezone-aware `datetime` objects (e.g., UTC) to avoid ambiguity.

3.3 Optimizations

- **Polling vs. Streaming:** The prompt mentioned "polling IBKR," but the implemented solution correctly uses IBKR's **streaming data** (`reqMktData`). Polling via `reqTickers` would introduce significant latency and is not suitable for a high-frequency validation task.

- **Data Structures:** For a very large universe of symbols, a simple dictionary might have performance implications. While unlikely to be a bottleneck, alternatives like a hash map implemented in C (via Cython) could be considered for extreme performance requirements.
- **Network Locality:** For lowest latency, the trading machine should be physically located in a data center with low-latency cross-connects to both ThetaData and IBKR's servers (e.g., Equinix NY4/NY5).
- **Code Profiling:** The `signal_processor` is the hot path. It should be profiled regularly to identify any performance bottlenecks. Logic should be kept minimal and computationally inexpensive. Vectorized operations with libraries like NumPy can be used if the alpha logic involves calculations on multiple ticks.